

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



Artificial Intelligence Lab: CSE 4618 **Lab 4 - Multi-Agent Search**

Written by - Namisa Najah Raisa
Student ID: 210042112

Sixth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

August 28, 2025

Contents

1	Introduction	3
2	Question 1: ReflexAgent	3
2.1	Problem Analysis	3
2.2	Solution	3
2.3	Findings	7
3	Question 2 : Minimax	8
3.1	Problem Analysis	8
3.2	Solution	9
3.2.1	Step 1: The value Function	11
3.2.2	Step 2: The max_value Function (Pacman's Turn)	11
3.2.3	Step 3: The min_value Function (Ghosts' Turn)	11
3.2.4	Step 4: Selecting the Best Action	12
3.3	Findings	12
4	Question 3: Alpha-Beta Pruning	13
4.1	Problem Analysis	13
4.2	Solution	13
4.3	Findings	16
5	Question 4: Expectimax	17
5.1	Problem Analysis	17
5.2	Solution	17
5.3	Findings	20
6	Question 5: Evaluation Function	20
6.1	Problem Analysis	20
6.2	Solution	20
6.3	Findings	24
7	Challenges	25

1 Introduction

In this lab, the task was to design intelligent agents for the classic Pacman game, focusing on both Pacman and ghost behaviors. The primary objectives included implementing adversarial search algorithms such as Minimax, Alpha-Beta pruning, and Expectimax, as well as experimenting with evaluation function design to improve decision-making.

2 Question 1: ReflexAgent

2.1 Problem Analysis

In this part of the lab, the task was to improve the **ReflexAgent** in **multiAgents.py** so that it can play more intelligently against ghosts while still efficiently collecting food. The provided ReflexAgent code demonstrated how to query the GameState for useful information such as food positions, ghost positions, and Pacman's current location. My main focus was to design a more effective evaluation function that assigns higher scores to favorable states. Specifically, the improved function should encourage Pacman to move closer to food pellets while avoiding dangerous positions near ghosts.

2.2 Solution

```
1 class ReflexAgent (Agent) :
2     """
3     A reflex agent chooses an action at each choice point by examining
4     its alternatives via a state evaluation function.
5
6     The code below is provided as a guide. You are welcome to change
7     it in any way you see fit, so long as you don't touch our method
8     headers.
9     """
10
11
12 def getAction(self, gameState):
13     """
14     You do not need to change this method, but you're welcome to.
15
16     getAction chooses among the best options according to the
17     evaluation function.
18
19     Just like in the previous project, getAction takes a GameState
20     and returns
21     some Directions.X for some X in the set {NORTH, SOUTH, WEST,
22         EAST, STOP}
23     """
24     # Collect legal moves and successor states
```

```

22     legalMoves = gameState.getLegalActions()
23
24     # Choose one of the best actions
25     scores = [self.evaluationFunction(gameState, action) for
26               action in legalMoves]
27     bestScore = max(scores)
28     bestIndices = [index for index in range(len(scores)) if
29                   scores[index] == bestScore]
30     chosenIndex = random.choice(bestIndices) # Pick randomly among
31               the best
32
33     "Add more of your code here if you want to"
34
35     return legalMoves[chosenIndex]
36
37 def evaluationFunction(self, currentGameState, action):
38     """
39     Design a better evaluation function here.
40
41     The evaluation function takes in the current and proposed
42     successor
43     GameStates (pacman.py) and returns a number, where higher
44     numbers are better.
45
46     The code below extracts some useful information from the
47     state, like the
48     remaining food (newFood) and Pacman position after moving
49     (newPos).
50     newScaredTimes holds the number of moves that each ghost will
51     remain
52     scared because of Pacman having eaten a power pellet.
53
54     Print out these variables to see what you're getting, then
55     combine them
56     to create a masterful evaluation function.
57     """
58     # Useful information you can extract from a GameState
59     (pacman.py)
60     successorGameState =
61         currentGameState.generatePacmanSuccessor(action)
62     newPos = successorGameState.getPacmanPosition()
63     newFood = successorGameState.getFood()
64     newGhostStates = successorGameState.getGhostStates()
65     newScaredTimes = [ghostState.scaredTimer for ghostState in
66                       newGhostStates]
67
68     *** YOUR CODE HERE ***

```

```

57     # 210042112
58
59     score = successorGameState.getScore()
60
61     foodList = newFood.asList()
62     if foodList:
63         minFoodDistance = min([manhattanDistance(newPos, food) for
64                                food in foodList])
65
66         score += 12 / (minFoodDistance + 1) # divisionByZero
67                                             # protection
68         # score += 1 / ((12 % minFoodDistance) + 1)
69
70     else:
71         score += 1212 # for clearing all the food
72
73     scaredGhosts = []
74     activeGhosts = []
75     for i, ghost in enumerate(newGhostStates):
76         if newScaredTimes[i] > 0:
77             scaredGhosts.append(ghost)
78         else:
79             activeGhosts.append(ghost)
80     if scaredGhosts:
81         closestScared =
82             min([manhattanDistance(newPos, g.getPosition()) for g in
83                 scaredGhosts])
84         score += 1/(closestScared + 1)
85         if closestScared == 0:
86             score += 112
87     if activeGhosts:
88         closestActive = min([manhattanDistance(newPos,
89                                                  g.getPosition()) for g in activeGhosts])
90         # if closestActive <= 1:
91         #     score -= 1212
92         if closestActive <= 12:
93             score -= 12 / (closestActive + 1)
94
95     if currentGameState.isWin():
96         score += 121212
97     if currentGameState.isLose():
98         score -= 121212
99
100    return score

```

Listing 1: Improved ReflexAgent

- I started with the provided **getAction** method. Here I didn't change much. It already

collects the legal moves, evaluates them using my **evaluationFunction**, and then picks one of the best moves. I just kept the random tie-break because it prevents Pacman from getting stuck in repetitive loops.

- In my **evaluationFunction**, the first thing I did was generate the **successorGameState** for the given action. This way I can check what happens **after** Pacman makes the move.
- From this successor state, I extracted the useful information: Pacman's new position (**newPos**), the food grid (**newFood**), the ghost states (**newGhostStates**), and the list of scared timers (**newScaredTimes**).
- I began the score with **successorGameState.getScore()**. This ensures that the game's built-in scoring is included as the base.
- Next, I handled the food. I converted the food grid into a list and calculated the Manhattan distance from Pacman to every pellet. I took the minimum of these distances so Pacman always cares about the **nearest** food. Then I added **12 / (minFoodDistance + 1)** to the score. The closer Pacman is to food, the bigger this bonus becomes. (And yes, I used 12 a lot because it's my ID number).
- If there was no food left at all, I added a large bonus (**+1212**) to strongly reward clearing the board.
- For ghosts, I split them into two groups: **scared ghosts** (if their scared timer is still positive) and **active ghosts** (if their timer is zero).
 - If there were scared ghosts, I encouraged Pacman to move towards them by adding **1 / (closestScared + 1)**. And if Pacman is actually on the same position as a scared ghost (**closestScared == 0**), I added a bigger reward of **112**.
 - If there were active ghosts, I checked the distance to the closest one. If it was less than or equal to 12, I subtracted **12 / (closestActive + 1)**. This discourages Pacman from walking into danger zones around ghosts.
- Finally, I handled terminal cases:
 - If the current state was a winning state, I added a very large reward (**+121212**).
 - If it was a losing state, I subtracted a very large penalty (**-121212**).

```

Question q1
=====

Pacman emerges victorious! Score: 1417
Pacman emerges victorious! Score: 1244
Pacman emerges victorious! Score: 1437
Pacman emerges victorious! Score: 1236
Pacman emerges victorious! Score: 1135
Pacman emerges victorious! Score: 1414
Pacman emerges victorious! Score: 1250
Pacman emerges victorious! Score: 1134
Pacman emerges victorious! Score: 1387
Pacman emerges victorious! Score: 1192
Average Score: 1284.6
Scores:      1417.0, 1244.0, 1437.0, 1236.0, 1135.0, 1414.0, 1250.0, 1134.0, 1387.0, 1192.0
Win Rate:    10/10 (1.00)
Record:      Win, Win, Win, Win, Win, Win, Win, Win, Win, Win
*** PASS: test_cases\q1\grade-agent.test (4 of 4 points)
***      1284.6 average score (2 of 2 points)
***      Grading scheme:
***      < 500:  0 points
***      >= 500:  1 points
***      >= 1000: 2 points
***      10 games not timed out (0 of 0 points)
***      Grading scheme:
***      < 10:  fail
***      >= 10:  0 points
***      10 wins (2 of 2 points)
***      Grading scheme:
***      < 1:  fail
***      >= 1:  0 points
***      >= 5:  1 points
***      >= 10: 2 points

### Question q1: 4/4 ###

Finished at 16:55:26

Provisional grades
=====
Question q1: 4/4
-----
Total: 4/4

```

Figure 1: Reflex Agent

2.3 Findings

By guiding Pacman towards the nearest food while avoiding ghosts, the agent already plays at a respectable level.

The agent strongly prioritized food collection. Pacman usually moved directly towards the nearest pellet and cleared small boards quite efficiently.

The use of randomness in tie-breaking helped the agent avoid repeating the same moves, but it also made the behavior slightly unpredictable between runs.

3 Question 2 : Minimax

3.1 Problem Analysis

The task is to implement an **MinimaxAgent** that can play Pacman using the minimax search algorithm. Unlike the simpler reflex agent, this agent must plan ahead by simulating future game states and making decisions that maximize Pacman's long-term score.

3.2 Solution

```
1 class MinimaxAgent(MultiAgentSearchAgent):
2     """
3     Your minimax agent (question 2)
4     """
5
6     def getAction(self, gameState):
7         """
8         Returns the minimax action from the current gameState using
9         self.depth
10        and self.evaluationFunction.
11
12        Here are some method calls that might be useful when
13        implementing minimax.
14
15        gameState.getLegalActions(agentIndex):
16        Returns a list of legal actions for an agent
17        agentIndex=0 means Pacman, ghosts are >= 1
18
19        gameState.generateSuccessor(agentIndex, action):
20        Returns the successor game state after an agent takes an action
21
22        gameState.getNumAgents():
23        Returns the total number of agents in the game
24
25        gameState.isWin():
26        Returns whether or not the game state is a winning state
27
28        gameState.isLose():
29        Returns whether or not the game state is a losing state
30        """
31        "*** YOUR CODE HERE ***"
32        # util.raiseNotDefined()
33        # 210042112
34
35        def value(state, agentIndex, depth):
36            if state.isWin() or state.isLose() or depth ==
37                self.depth: # win/lose/max search depth
38                return self.evaluationFunction(state)
39
40            if agentIndex == 0: # pacman
41                return max_value(state, agentIndex, depth)
42            else:
43                return min_value(state, agentIndex, depth)
44
45        def max_value(state, agentIndex, depth):
46            v = float('-inf') # start with the worst case so I can
47                replace it with the best case
```

```

43     if state.getLegalActions(agentIndex) == 0:
44         return self.evaluationFunction(state)
45
46     else:
47
48         for action in state.getLegalActions(agentIndex):
49             successor = state.generateSuccessor(agentIndex,
50                 action)
51             # Next agent, same depth if not Pacman, else
52             increase depth
53             nextAgent = (agentIndex + 1) % state.getNumAgents()
54             nextDepth = depth + 1 if nextAgent == 0 else depth
55             v = max(v, value(successor, nextAgent, nextDepth))
56         return v
57
58 def min_value(state, agentIndex, depth):
59     v = float('inf')
60     if state.getLegalActions(agentIndex) == 0:
61         return self.evaluationFunction(state)
62     else:
63
64         for action in state.getLegalActions(agentIndex):
65             successor = state.generateSuccessor(agentIndex,
66                 action)
67             nextAgent = (agentIndex + 1) % state.getNumAgents()
68             nextDepth = depth + 1 if nextAgent == 0 else depth
69             v = min(v, value(successor, nextAgent, nextDepth))
70         return v
71
72 legalActions = gameState.getLegalActions(0)
73 bestScore = float('-inf')
74 bestAction = None
75
76 for action in legalActions:
77     successor = gameState.generateSuccessor(0, action)
78     score = value(successor, 1, 0)
79     if score > bestScore:
80         bestScore = score
81         bestAction = action
82
83 return bestAction

```

Listing 2: Minimax Agent

First, I defined the **getAction** method, which returns the best action Pacman should take using the Minimax algorithm. This method uses a helper function called **value** to recursively evaluate states.

3.2.1 Step 1: The value Function

I created the **value(state, agentIndex, depth)** function to handle the recursive minimax evaluation. It takes the current game state, the index of the agent (Pacman or a ghost), and the current depth in the search tree.

Inside this function:

- If the state is a win, lose, or the depth equals the maximum depth (**self.depth**), I return the evaluation function score because that's the base case where we stop searching.
- If the agent is Pacman (**agentIndex = 0**), I call **max_value** because Pacman is trying to maximize the score. Otherwise, I call **min_value** because ghosts are minimizing Pacman's score.

3.2.2 Step 2: The max_value Function (Pacman's Turn)

In **max_value(state, agentIndex, depth)**, I initialize:

$$v = -\infty$$

because Pacman wants the maximum value.

- If there are no legal actions for Pacman, I return the evaluation of the current state.
- Otherwise, I loop through all legal actions, generate the successor state for each action, and compute its minimax value by calling **value** on the next agent.
- The next agent is computed as:

$$\text{nextAgent} = (\text{agentIndex} + 1) \bmod \text{state.getNumAgents}()$$

- If the next agent is Pacman again, I increase the depth by 1; otherwise, the depth stays the same.
- Finally, I update:

$$v = \max(v, \text{value}(\text{successor}, \text{nextAgent}, \text{nextDepth}))$$

3.2.3 Step 3: The min_value Function (Ghosts' Turn)

For **min_value(state, agentIndex, depth)**, I do the opposite:

$$v = +\infty$$

because the ghost wants to minimize Pacman's score.

- If there are no legal actions for the ghost, I return the evaluation score.
- Otherwise, I loop over all legal actions for the ghost, generate successors, and recursively call **value** for the next agent, updating:

$$v = \min(v, \text{value}(\text{successor}, \text{nextAgent}, \text{nextDepth}))$$

3.2.4 Step 4: Selecting the Best Action

Finally, in **getAction**, I look at all legal actions for Pacman from the current state:

- For each action, I generate the successor state and call **value** starting with the first ghost (**agentIndex** = 1) and depth 0.
- I track the action that gives the highest score and return that as the best action for Pacman.

I followed the minimax principle where Pacman tries to maximize its score, and ghosts try to minimize it. I also accounted for multiple ghosts by cycling through agent indexes using modulus and only increasing depth after all agents have moved. This ensures the tree expands level by level up to the given search depth.

```
Pacman died? Score: 84
Average Score: 84.0
Scores:      84.0
Win Rate:    0/1 (0.00)
Record:      Loss
*** Finished running MinimaxAgent on smallClassic after 2 seconds.
*** Won 0 out of 1 games. Average score: 84.000000 ***
*** PASS: test_cases\q2\8-pacman-game.test

### Question q2: 5/5 ###

Finished at 16:56:01

Provisional grades
=====
Question q2: 5/5
-----
Total: 5/5
```

Figure 2: Minimax Agent

3.3 Findings

The Minimax algorithm indicates that Pacman navigates through the game tree considering both its own actions and those of the ghost agents. The Pacman maximize its own score and minimize the ghost's scores. Additionally, the Minimax algorithm successfully handles both terminal states and intermediate states.

4 Question 3: Alpha-Beta Pruning

4.1 Problem Analysis

This task was to implement a new agent, called AlphaBetaAgent, which uses alpha-beta pruning to efficiently explore the Minimax tree in a multi-agent environment. The goal was to optimize the standard Minimax search by pruning branches that cannot affect the final decision, thereby reducing unnecessary computation.

4.2 Solution

```
1  class AlphaBetaAgent(MultiAgentSearchAgent):
2      """
3      Your minimax agent with alpha-beta pruning (question 3)
4      """
5
6  def getAction(self, gameState):
7      """
8      Returns the minimax action using self.depth and
9      self.evaluationFunction
10     """
11     """*** YOUR CODE HERE ***"""
12     # util.raiseNotDefined()
13     # 210042112
14     def alphaBeta(agentIndex, depth, gameState, alpha, beta):
15         if gameState.isWin() or gameState.isLose() or depth ==
16             self.depth:
17             return self.evaluationFunction(gameState)
18
19         if agentIndex == 0:
20             return maxValue(agentIndex, depth, gameState, alpha,
21                             beta)
22         else:
23             return minValue(agentIndex, depth, gameState, alpha,
24                             beta)
25
26     def maxValue(agentIndex, depth, gameState, alpha, beta):
27         actions = gameState.getLegalActions(agentIndex)
28         if not actions:
29             return self.evaluationFunction(gameState)
30
31         v = float('-inf')
32
33         for action in actions:
```

```

33         nextAgent = (agentIndex + 1) % gameState.getNumAgents()
34         nextDepth = depth + 1 if nextAgent == 0 else depth
35         successor = gameState.generateSuccessor(agentIndex,
36             action)
37         v = max(v, alphaBeta(nextAgent, nextDepth, successor,
38             alpha, beta))
39         if v > beta:
40             return v # Beta pruning
41         alpha = max(alpha, v)
42
43     return v
44
45 def minValue(agentIndex, depth, gameState, alpha, beta):
46     actions = gameState.getLegalActions(agentIndex)
47     if not actions:
48         return self.evaluationFunction(gameState)
49
50     v = float('inf')
51
52     for action in actions:
53         nextAgent = (agentIndex + 1) % gameState.getNumAgents()
54         nextDepth = depth + 1 if nextAgent == 0 else depth
55         successor = gameState.generateSuccessor(agentIndex,
56             action)
57         v = min(v, alphaBeta(nextAgent, nextDepth, successor,
58             alpha, beta))
59         if v < alpha:
60             return v # Alpha pruning
61         beta = min(beta, v)
62     return v
63
64     alpha = float('inf')
65     beta = float('inf')
66
67     actions = gameState.getLegalActions(0)
68     bestAction = None
69     bestValue = float('inf')
70
71     for action in actions:
72         successor = gameState.generateSuccessor(0, action)
73         value = alphaBeta(1, 0, successor, alpha, beta)
74         if value > bestValue:
75             bestValue = value
76             bestAction = action
77         alpha = max(alpha, value)
78
79     return bestAction

```

Listing 3: Alpha-Beta Agent

In this task, I implemented the **AlphaBetaAgent** to perform Minimax search with alpha-beta pruning in a multi-agent environment.

1. **Alpha-Beta Recursive Function:** I defined a helper function **alphaBeta(agentIndex, depth, gameState, alpha, beta)** which recursively computes the Minimax value of a game state.
 - At the beginning of each call, I checked if the current state is a terminal state (win or lose) or if the search has reached the maximum depth (**self.depth**). If so, I return the evaluation of the current game state using **self.evaluationFunction(gameState)**. This ensures the recursion stops at leaves of the tree or when the depth limit is reached.
 - If it is Pacman's turn (**agentIndex == 0**), I call the **maxValue** function. Otherwise, I call the **minValue** function for ghost agents.
2. **Max Function (maxValue):** This function calculates the best score Pacman can achieve from the current state.
 - I initialize **v** to negative infinity to represent the worst-case scenario for **MAX**.
 - For each legal action Pacman can take, I compute the successor state and determine the **nextAgent** and **nextDepth**. The depth is incremented when all agents have played a turn.
 - I recursively call **alphaBeta** on the successor state and update **v** as the maximum of the current value and the returned value.
 - If **v ≥ beta**, I perform **beta pruning**, because **MIN** will avoid this branch, so further exploration is unnecessary.
 - I update **alpha = max(alpha, v)** to maintain the best value MAX can guarantee along the current path.
3. **Min Function (minValue):** This function calculates the best score the ghost (MIN) can enforce.
 - I initialize **v** to positive infinity, representing the worst-case scenario for **MIN**.
 - For each legal action of the ghost, I compute the successor state, **nextAgent**, and **nextDepth** as in **maxValue**.
 - I recursively call **alphaBeta** and update **v** as the minimum of the current value and the returned value.
 - If **v ≤ alpha**, I perform **alpha pruning**, because **MAX** will never choose this branch, so further exploration is unnecessary.
 - I update **beta = min(beta, v)** to maintain the best value MIN can enforce along the current path.

4. Root-Level Action Selection:

- At the root (Pacman's turn), I initialize **alpha** to negative infinity and **beta** to positive infinity.
- I iterate over all of Pacman's legal actions and compute the Minimax value for each action using **alphaBeta**.
- I track the best value (**bestValue**) and the corresponding action (**bestAction**) for Pacman.
- I update **alpha** = **max(alpha, value)** after each action to propagate the best value found so far to subsequent branches for pruning.
- Finally, I return **bestAction** as the optimal move for Pacman.

```
Average Score: 84.0
Scores:        84.0
Win Rate:      0/1 (0.00)
Record:        Loss
*** Finished running AlphaBetaAgent on smallClassic after 2 seconds.
*** Won 0 out of 1 games. Average score: 84.000000 ***
*** PASS: test_cases\q3\8-pacman-game.test

### Question q3: 5/5 ###

Finished at 16:56:45

Provisional grades
=====
Question q3: 5/5
-----
Total: 5/5
```

Figure 3: Alpha-beta Agent

4.3 Findings

The alpha-beta pruning leads to improvements compared to traditional Minimax search. By pruning branches of the search tree that will not affect the final decision, the algorithm reduces the number of nodes explored. This leads to faster computation and more optimal decision-making.

5 Question 4: Expectimax

5.1 Problem Analysis

This task was to implement an ExpectimaxAgent that models opponents' probabilistic behavior by computing expected values of their actions.

5.2 Solution

```
1  class ExpectimaxAgent(MultiAgentSearchAgent):
2      """
3      Your expectimax agent (question 4)
4      """
5
6  def getAction(self, gameState):
7      # """
8      # Returns the expectimax action using self.depth and
9      # self.evaluationFunction
10
11     # All ghosts should be modeled as choosing uniformly at random
12     # from their
13     # legal moves.
14     # """
15
16     """*** YOUR CODE HERE ***"""
17     # util.raiseNotDefined()
18     # 210042112
19
20     def expectimax(agentIndex, depth, gameState):
21         if gameState.isWin() or gameState.isLose() or depth ==
22             self.depth:
23             return self.evaluationFunction(gameState)
24
25         if agentIndex == 0:
26             return maxValue(agentIndex, depth, gameState)
27         else:
28             return expValue(agentIndex, depth, gameState)
29
30     def maxValue(agentIndex, depth, gameState):
31         v = float('-inf')
32         if gameState.getLegalActions(agentIndex) == 0:
33             return self.evaluationFunction(gameState)
34         else:
35             for action in gameState.getLegalActions(agentIndex):
36                 successor =
37                     gameState.generateSuccessor(agentIndex, action)
38                 nextAgent = (agentIndex + 1) %
39                     gameState.getNumAgents()
40                 nextDepth = depth + 1 if nextAgent == 0 else depth
```

```

34         v = max(v, expectimax(nextAgent, nextDepth,
35                               successor))
36     return v
37
38 def expValue(agentIndex, depth, gameState):
39     v = 0
40     if gameState.getLegalActions(agentIndex) == 0:
41         return self.evaluationFunction(gameState)
42     else:
43         for action in gameState.getLegalActions(agentIndex):
44             nextAgent = (agentIndex + 1) %
45                         gameState.getNumAgents()
46             nextDepth = depth + 1 if nextAgent == 0 else depth
47             successor =
48                 gameState.generateSuccessor(agentIndex, action)
49             p = 1.0 /
50                 len(gameState.getLegalActions(agentIndex)) #
51                 uniform probability for this successor
52             v += p * expectimax(nextAgent, nextDepth,
53                               successor)
54
55     return v
56
57 actions = gameState.getLegalActions(0)
58 bestAction = None
59 bestValue = float('-inf')
60
61 for action in actions:
62     successor = gameState.generateSuccessor(0, action)
63     value = expectimax(1, 0, successor)
64     if value > bestValue:
65         bestValue = value
66         bestAction = action
67
68 return bestAction

```

Listing 4: Expectimax Agent

- **I defined a recursive function `expectimax(agentIndex, depth, gameState)`:** I did this to unify the traversal of the multi-agent tree in one place. It decides whether to stop (base case) or delegate to a max node (Pacman) or a chance node (ghost).
- **I return the evaluation at terminal or depth limit:** I did this because we need a numeric utility at leaves of the search tree; stopping either when the game ends or when the allowed search depth is reached prevents infinite recursion and bounds the computation.
- **I implemented `maxValue` for Pacman:** I did this because Pacman is modeled

as a rational agent. For each legal Pacman action I generate the successor state, compute the expectimax value of that child (starting from the next agent), and take the maximum across all actions. This yields the best achievable outcome for Pacman given the model of subsequent agents.

- **I implemented expValue for ghosts (chance nodes):** I did this because ghosts are modeled as stochastic (uniformly random) actors. For each ghost action I generate the successor, call expectimax on that successor, multiply by the action probability (uniform $1/|\text{actions}|$), and accumulate the weighted sum. This produces the expected utility of the ghost's move.
- **I handle the agent cycling and depth increment carefully:** I did this because multiple ghosts exist. After computing `nextAgent = (agentIndex + 1) % gameState.getNumAgents()`, I only increment the `depth` when we cycle back to Pacman (i.e., when `nextAgent == 0`). This way one full ply corresponds to a round where every agent has moved once.
- **I check for empty legal action lists:** I did this to be robust: if an agent has no legal moves I return the evaluation of the current state rather than attempting to iterate an empty action list, which avoids errors and matches the terminal behavior.
- **I selected the root action by iterating Pacman actions and comparing expectimax values:** I did this because `getAction` must return an actual move. For each Pacman action I simulate the successor and ask the tree (starting from ghost 1) what the expected utility is; the action with the greatest utility is returned.

I implemented an expectimax search that treats Pacman as a rational maximizer and ghosts as uniformly random agents. I structured the code to (1) detect leaf/terminal states, (2) correctly alternate agents while updating depth after a full round, (3) compute max for Pacman and expected values for ghosts, and (4) select the root action with the highest computed utility. This matches the expectimax specification and is robust to edge cases like no-legal-action states.

```

Average Score: 84.0
Scores:      84.0
Win Rate:    0/1 (0.00)
Record:      Loss
*** Finished running ExpectimaxAgent on smallClassic after 2 seconds.
*** Won 0 out of 1 games. Average score: 84.000000 ***
*** PASS: test_cases\q4\7-pacman-game.test

### Question q4: 5/5 ###

Finished at 16:57:08

Provisional grades
=====
Question q4: 5/5
-----
Total: 5/5

```

Figure 4: Expectimax Agent

5.3 Findings

The implementation of the Expectimax algorithm introduces a probabilistic element to the decision-making process by calculating the expected value based on the probabilities of different actions.

6 Question 5: Evaluation Function

6.1 Problem Analysis

This task was to design and implement an improved evaluation function for Pacman in the betterEvaluationFunction from multiAgents.py. Unlike the reflex agent that evaluates actions, this function should evaluate game states by considering factors such as food, ghosts, scared timers, and win/lose conditions to guide Pacman's strategic decision-making.

6.2 Solution

```

1  def betterEvaluationFunction(currentGameState):
2      """
3      Your extreme ghost-hunting, pellet-nabbing, food-gobbling,
4      unstoppable
5      evaluation function (question 5).
6
7      DESCRIPTION: <write something here so we know what you did>

```

```

7      """
8      "*** YOUR CODE HERE ***"
9      # util.raiseNotDefined()
10     #210042112
11     score = currentGameState.getScore()
12     position = currentGameState.getPacmanPosition()
13     ghostStates = currentGameState.getGhostStates()
14     Food = currentGameState.getFood()
15     scaredTimes = [ghost.scaredTimer for ghost in ghostStates]
16
17     foodList = Food.asList()
18     if foodList:
19         foodDistances = [manhattanDistance(position, food) for food in
20             foodList]
21         closestFood = min(foodDistances)
22         score += 12 / (closestFood + 1)
23     else:
24         score += 1212
25
26     scaredGhosts = []
27     activeGhosts = []
28     for i, ghost in enumerate(ghostStates):
29         if scaredTimes[i] > 0:
30             scaredGhosts.append(ghost)
31         else:
32             activeGhosts.append(ghost)
33
34     if scaredGhosts:
35         closestScared = min([manhattanDistance(position,
36             g.getPosition()) for g in scaredGhosts])
37         score += 1 / (closestScared + 1)
38         if closestScared == 0:
39             score += 112
40     if activeGhosts:
41         closestActive = min([manhattanDistance(position,
42             g.getPosition()) for g in activeGhosts])
43         # if closestActive <= 1:
44         #     score -= 1212
45         if closestActive <= 12 :
46             score -= 12 / (closestActive + 1)
47
48     if currentGameState.isWin():
49         score += 121212
50     if currentGameState.isLose():
51         score -= 121212

```

Listing 5: Better Evaluation Function

1. I got `currentGameState.getScore()`, Pac-Man's position via `getPacmanPosition()`, the list of ghosts via `getGhostStates()`, and the food grid via `getFood()`. Because the evaluation must reflect the current situation, so starting from the official game score and positional data gives the function grounded inputs for every decision.
2. **I extracted ghost scared timers.**
I created `scaredTimes = [ghost.scaredTimer for ghost in ghostStates]`. Because ghosts behave differently when scared, I need their timers to decide whether to encourage chasing or to avoid them.
3. **I converted the food grid to a list and scored proximity.**
I wrote `foodList = Food.asList()` and if there is food I computed Manhattan distances to each food pellet, took the minimum as `closestFood`, and added `12 / (closestFood + 1)` to the score; otherwise I added a large constant (`1212`).
Because I want Pac-Man to prefer states where the nearest food is closer. Using a reciprocal $\frac{12}{d+1}$ gives diminishing returns the farther the food is (and avoids division by zero). The large constant for “no food” rewards states that are essentially cleared of pellets.
4. **I separated ghosts into scared and active lists.**
I looped over ghosts with `for i, ghost in enumerate(ghostStates):` and appended each ghost to either `scaredGhosts` or `activeGhosts` depending on `scaredTimes[i] > 0`.
Because different scoring rules apply: scared ghosts are targets to chase, while active ghosts are threats to avoid.
5. **I encouraged chasing scared ghosts (safely).**
If `scaredGhosts` is non-empty I computed the Manhattan distance to the closest scared ghost and increased the score by `1 / (closestScared + 1)`; if Pac-Man is on the same tile (`closestScared == 0`) I added a capture bonus of `112`.
Because being near a scared ghost is good (opportunity to capture and gain points), so the function slightly prefers states that close that distance; the capture bonus incentivizes actually getting the ghost.
6. **I penalized proximity to active (dangerous) ghosts.**
If there are active ghosts I computed `closestActive` and, when it is within a defined caution range (here ≤ 12), I subtracted `12 / (closestActive + 1)` from the score.
Because closer active ghosts increase risk of death; the reciprocal penalty scales stronger when ghosts are nearer, which pushes Pac-Man away from dangerous situations.
7. **I handled terminal conditions strongly.**
I added a large positive bonus `+121212` for a win and a large negative penalty `-121212` for a loss.

Because terminal states should dominate the heuristic: winning must be treated as extremely good and losing as extremely bad so search prefers paths to victory and avoids losing ones.

8. **I returned the combined score.**

Finally, after combining the base score and the adjustments above, I returned **score**.

I wrote the evaluation to be state-based (not action-based) so it can be used by multi-agent search algorithms that score states at leaf nodes. Each line in the implementation intentionally maps to a design decision: encourage food, opportunistically chase scared ghosts, avoid active ghosts, and strongly prefer terminal victories.

```

Question q5
=====

Pacman emerges victorious! Score: 1162
Pacman emerges victorious! Score: 1034
Pacman emerges victorious! Score: 1096
Pacman emerges victorious! Score: 967
Pacman emerges victorious! Score: 1346
Pacman emerges victorious! Score: 1219
Pacman emerges victorious! Score: 1145
Pacman emerges victorious! Score: 1174
Pacman emerges victorious! Score: 1259
Pacman emerges victorious! Score: 774
Average Score: 1117.6
Scores:      1162.0, 1034.0, 1096.0, 967.0, 1346.0, 1219.0, 1145.0, 1174.0, 1259.0, 774.0
Win Rate:    10/10 (1.00)
Record:      Win, Win, Win, Win, Win, Win, Win, Win, Win, Win
*** PASS: test_cases\q5\grade-agent.test (6 of 6 points)
***      1117.6 average score (2 of 2 points)
***      Grading scheme:
***          < 500:  0 points
***          >= 500:  1 points
***          >= 1000: 2 points
***      10 games not timed out (1 of 1 points)
***      Grading scheme:
***          < 0:  fail
***          >= 0:  0 points
***          >= 10:  1 points
***      10 wins (3 of 3 points)
***      Grading scheme:
***          < 1:  fail
***          >= 1:  1 points
***          >= 5:  2 points
***          >= 10: 3 points

### Question q5: 6/6 ###

Finished at 16:58:09

Provisional grades
=====
Question q5: 6/6
-----
Total: 6/6

```

Figure 5: Better Evaluation Function

6.3 Findings

The evaluation function considers Pacman's current game score, the distance to the nearest food, and the states of ghosts (scared or active). It rewards Pacman for moving closer to food and scared ghosts while giving larger bonuses for eating them. At the same time, it penalizes Pacman for being too close to active ghosts, encouraging safer paths. Additionally, large rewards are given for winning states and strong penalties for losing states.

7 Challenges

It took me a lot of time to design the evaluation functions. Furthermore, I couldn't fully understand how the pseudocode of the alpha-beta agent was supposed to be implemented. But taking help from the Minimax agent solved the problem.